HyperPod

Measured Placement and Failure Recovery for Meta Llama Models on a Pool of Consumer Laptops

Technical Report

Dharshaneshwaran
Meta Global AI Developer Hackathon — Indie Developer Track

31 October 2026

Abstract

A student or independent developer may own several modest laptops and still have no single GPU large enough for the local model they want to run. `llama.cpp` can already expose a remote accelerator over its RPC backend, so the memory of several machines can be combined. That backend is documented upstream as a proof of concept: when one RPC peer disappears, the process that was using it aborts. Consumer laptops disappear often, which makes the combined pool difficult to depend on.

This report documents HyperPod, a control plane around `llama.cpp` that discovers laptops on a local network, records the hardware each one actually reports, derives tensor weights from measured free video memory, supervises the model process, and rebuilds the pipeline when a worker stops answering. It reports what we measured on three physical machines: a Linux laptop with an RTX 3050 acting as controller, a Windows laptop with an RTX 4050, and an M4 MacBook Air running Metal, all on one Wi-Fi network.

Three results are quantified. First, a Llama 3.2 3B model was loaded across all three machines, and the per-device byte counts reported by `llama.cpp` are given. Second, the Mac accelerator was stopped during a request: the underlying process aborted, HyperPod excluded that endpoint, rebuilt across the two surviving laptops in 33 s, and the same HTTP request returned. Under identical conditions, unmodified `llama.cpp` failed the request and did not restart. Third, splitting over Wi-Fi was substantially slower than the single local GPU, at a median of 11.8 s against 1.54 s per request. HyperPod is therefore a capacity and recovery mechanism, not an accelerator, and this report states that plainly.

Contents

1 Introduction **3**
1.1 What this report covers . . . 3
1.2 What this report does not claim . . . 3

2 Background: the failure we had to handle **4**
2.1 How the RPC backend distributes a model . . . 4
2.2 What happens when a worker disappears . . . 4
2.3 The resulting requirement . . . 5

3 Design **5**
3.1 Components . . . 5
3.2 Telemetry that refuses to guess . . . 5
3.3 Deriving the split from measurement . . . 6
3.4 Choosing the failure signal . . . 6

4 Implementation **6**
4.1 Go controller . . . 7
4.2 Rust worker . . . 7
4.3 Dashboard . . . 7
4.4 Tests . . . 7

5 Experimental setup **8**
5.1 Machines . . . 8
5.2 Software . . . 8
5.3 Models . . . 8
5.4 Network . . . 8
5.5 Method . . . 8

6 Results **9**
6.1 Measured telemetry . . . 9
6.2 Scheduling and actual placement . . . 9
6.3 Latency and throughput . . . 10
6.4 Failure and recovery . . . 11
6.5 Load times . . . 12

7 Limitations **12**

8 Related work **13**

9 Conclusion **14**

1 Introduction

The hardware a student actually owns is rarely the hardware the tooling assumes. Over a few years a person may accumulate two or three laptops, each adequate on its own terms, none of them carrying a large GPU. The machines used throughout this report are a case in point: a Linux laptop with an RTX 3050 exposing 4,096 MiB of video memory, a Windows laptop with an RTX 4050 exposing 6,141 MiB, and an M4 MacBook Air with 16,384 MiB of unified memory. Individually, none of them holds a 4.9 GB quantised model in GPU memory. Together they hold more than enough.

`llama.cpp` already provides the mechanism for combining them. Its RPC backend exposes a remote accelerator over TCP, and the main process can offload model layers to it. The upstream documentation is explicit that this backend is a proof of concept and that it should not be exposed to untrusted networks. It is equally explicit in behaviour: if an RPC peer stops responding, the process using it aborts. We reproduced this directly, and the error text is quoted in Section 2.

That behaviour is the obstacle. A laptop is not a server. Its lid gets closed, its Wi-Fi roams, its owner stops a terminal. Each of those events ends the inference process outright, and the model must be loaded again from the beginning. On the network used here, reloading a 3B model across two machines took between 33 s and 240 s depending on cache state, so an unhandled failure is not a brief interruption.

1.1 What this report covers

HyperPod is the control plane we built around `llama.cpp` to make that pool usable. It does not implement attention kernels, quantisation, or device offload; `llama.cpp` performs all of that. HyperPod is responsible for five things:

1. finding worker laptops on the local network without manual addresses;
2. recording what each laptop reports about its own hardware, and marking anything it cannot report as unavailable rather than guessing;
3. deriving tensor weights from measured free video memory;
4. reading back the placement `llama.cpp` actually chose, rather than assuming the requested split was honoured;
5. detecting that the model process has died, excluding the endpoint responsible, rebuilding on what remains, and retrying the request that was interrupted.

Section 3 describes the design, including the one scheduling decision that required a measurement to settle. Section 6 reports results from the three machines above: the placement `llama.cpp` produced, five-run latency and throughput figures against two baselines, and two physical failure experiments in which an accelerator was stopped by hand during a live request.

1.2 What this report does not claim

Capability-aware placement across heterogeneous devices is not new, and neither is failure recovery in distributed serving. Both exist in datacentre systems and in prior work on consumer-hardware inference, discussed in Section 8. The contribution here is narrower: a working, measured implementation on three mismatched laptops across CUDA and Metal, with every claim backed by a committed log, and with the costs reported rather than omitted.

Three specific costs are reported in full. Splitting over Wi-Fi was roughly six times slower per output token than the single local GPU. Recovery restarts the model process and re-runs the request; it does not resume the KV cache or continue from the token where the failure occurred.

And the 8B configuration demonstrates pooled capacity only: because that model does not fit in the controller's GPU memory alone, the two-machine arrangement cannot survive the loss of the remote worker. Each of these is stated again in Section 7 so that no reader has to infer them.

2 Background: the failure we had to handle

2.1 How the RPC backend distributes a model

Built with `-DGGML_RPC=ON`, `llama.cpp` provides `ggml-rpc-server`, which advertises the accelerators of the machine it runs on. A client passes `-rpc host:port` and those remote devices join the local ones as offload targets. Model layers are then distributed across the combined set, and `-tensor-split` can override the default proportions.

Two properties of this mechanism shaped our design.

First, the device order used by `-tensor-split` is not the order printed by `-list-devices`. We established the real order by measurement rather than by reading documentation, loading the same 3B model twice with opposite weights:

`-tensor-split`	ROG CUDA0	TUF RPC0
`1,4`	1,573 MiB	345 MiB
`4,1`	552 MiB	1,366 MiB

Weighting the first position moved memory onto the local GPU, so RPC devices are listed first and the local device last. Our initial implementation had this backwards, which placed most of the model on the controller while the logs claimed it was distributed. Reading the placement back from `llama.cpp`, rather than trusting the request, is what exposed the error.

Second, the RPC server accepts one client at a time. While a model is resident, the device cannot be queried by a second process. We confirmed this when a device probe aborted against an endpoint already held by the running model. This rules out querying remote devices for live memory figures during serving, and it is why worker-reported telemetry is the only usable source.

2.2 What happens when a worker disappears

The behaviour on peer loss is unambiguous. With a 3B model split across the Linux and Windows laptops, we began a request and then stopped the remote `ggml-rpc-server` by hand. `llama.cpp` emitted:

```
Remote RPC server crashed or returned malformed response
```

and the process aborted. Measured from the client side: `curl` exited 52 with HTTP status 000 after 34.06 s, no response body was produced, the process was gone five seconds later, the health endpoint was unreachable, and nothing restarted. The model had to be loaded again from scratch.

This is reasonable behaviour for a proof-of-concept backend. A lost peer means part of the computation graph is unreachable mid-forward-pass, and aborting is safer than continuing with an incomplete model. The consequence, though, is that the reliability of the whole pool equals the reliability of its least stable member. For laptops on Wi-Fi, that is low.

2.3 The resulting requirement

Recovery cannot be implemented inside the forward pass without modifying `llama.cpp` itself, which we chose not to do: it would fork a fast-moving upstream project and make the result harder to reproduce. The alternative is to treat the model process as replaceable and to supervise it from outside. That places three obligations on the supervisor:

- **Distinguish failure modes.** An ordinary bad request must not trigger a rebuild, and a dead worker must not be mistaken for one. We settled on process exit as the signal, for reasons given in Section 3.
- **Verify endpoints independently of agent health.** A worker's heartbeat proves its agent is running, not that its accelerator is reachable. The experiment in Section 6.4 was designed specifically to separate these two states.
- **Preserve the interrupted request.** A user who submitted a prompt before the failure should not have to resubmit it.

3 Design

3.1 Components

HyperPod has three parts: a Go controller on the machine the user sits at, a Rust agent on every laptop that contributes compute, and a React dashboard. Figure 1 shows how they relate to the `llama.cpp` processes that perform the actual inference.

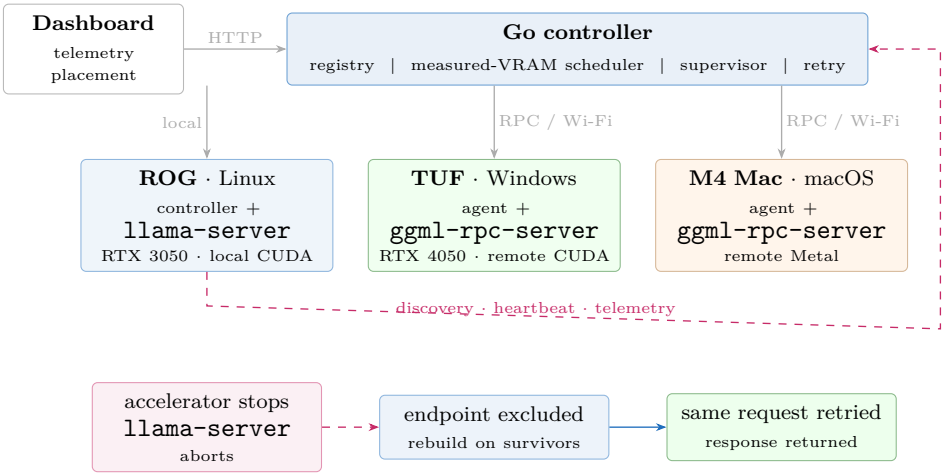


Figure 1: HyperPod components and the recovery path. `llama.cpp` performs all inference; the Go controller decides placement, observes it, and replaces the model process when a worker stops answering. The bottom row is the sequence measured in Section 6.4.

3.2 Telemetry that refuses to guess

Each agent samples its own hardware every five seconds and reports the snapshot with registration, with every heartbeat, and in its discovery broadcast. The schema is versioned, and the controller rejects a version it does not understand.

The design rule that mattered most is that every value which can be unavailable is nullable, and an unavailable value is transmitted as `null` rather than as zero or as a plausible substitute. Three cases from our own hardware show why:

- The Windows RTX 4050 reports no GPU power limit; `nvidia-smi` returns `[N/A]`. That field stays `null`.
- Apple Silicon has one unified memory pool, so there is no separate free-video-memory quantity. Reporting system RAM in that field would imply a dedicated figure that does not exist, so it stays `null`.
- macOS exposes no direct equivalent of `MemAvailable`. We derive available memory from `vm_stat` page classes and name that source explicitly, so a reader can judge the estimate.

Sources are carried in the payload (`nvidia-smi`, `windows-cim`, `macos-sysctl+vm_stat`, and so on) and shown in the dashboard, so no number in the interface is anonymous.

3.3 Deriving the split from measurement

When every selected device reports free video memory, the controller computes

$$w_i \;=\; \text{free_vram}_i \;-\; R, \qquad R = 512\,\text{MiB},$$

where R is a fixed reserve held back on each device for runtime buffers, and the resulting w_i are passed to `-tensor-split` in RPC-first order. Endpoints are sorted before the weights are built, so the same measurements always produce the same argument string. R is a policy choice, not a measurement, and it is labelled as such in the interface.

If any required value is missing — as with the Mac — the controller supplies no weights at all and records the strategy as `llama.cpp-auto (remote free VRAM unavailable)`. `llama.cpp` then probes the devices itself. We preferred this to deriving a weight from a GPU name or from total memory, because a wrong weight produces a silently unbalanced model rather than a visible gap.

3.4 Choosing the failure signal

Detecting worker loss was the hardest design decision, and our first two attempts were wrong.

Port probing was too eager: a TCP connection can succeed against a socket whose server is no longer serving, and it can fail transiently on a busy Wi-Fi link. Health polling was no better, because a loaded model can be slow to answer while remaining healthy.

The signal we settled on is the exit of the supervised `llama-server` process. When an RPC peer is lost, `llama.cpp` aborts, so the process exit is both reliable and specific. Two refinements were needed:

- **Loading is not failure.** A model still loading answers health checks with 503. An early version treated that as death and killed its own load. Readiness is now watched by a separate observer with no timeout, and a request that arrives during loading waits and is then told the model is still loading.
- **The abort and the client error race.** The HTTP request fails milliseconds before the process exit is observed. A failed request therefore waits up to five seconds to see whether the process exits before it is classified as an ordinary error.

On rebuild, every candidate endpoint is probed directly. A worker whose heartbeat is healthy but whose accelerator is unreachable is excluded, which is exactly the condition tested in Section 6.4.

4 Implementation

The repository contains roughly 3,570 lines of our own code across three languages, excluding the vendored `llama.cpp` checkout and generated dependency files.

4.1 Go controller

The controller owns cluster state and the model process. It exposes `POST /register`, `POST /heartbeat`, `GET /status`, `GET /pipeline` and `POST /infer`, and listens for discovery broadcasts on UDP 50999. A worker is considered healthy if its last heartbeat arrived within ten seconds.

Two implementation details are worth recording because both were bugs first.

The placement reader parses `llama.cpp`'s own load output. Running the model server at log verbosity 4 makes it print one line per device of the form `load_tensors: <device> model buffer size = <N> MiB`, which the controller captures into structured per-device figures. This is the only reason we noticed the reversed tensor-split order described in Section 2: the requested weights and the actual placement disagreed.

The process is started from the directory holding the `llama.cpp` binaries. Renaming the project directory broke the build until we fixed this, because the server resolves its shared libraries relative to its own location and exited with status 127 before contacting any worker.

4.2 Rust worker

One agent binary serves Windows, Linux and macOS. It broadcasts presence, registers, heartbeats every three seconds, and re-registers automatically if a heartbeat fails, which is what allows the cluster to reassemble after the controller restarts.

A single sampler runs every five seconds and its snapshot is shared by all three message paths. This matters on Windows, where each sample would otherwise spawn separate PowerShell and `nvidia-smi` processes for registration, heartbeats and discovery.

Table 1: Telemetry sources per platform. Every value these tools decline to report is transmitted as `null`.

	GPU	**Memory**	**Power**
Linux (NVIDIA)	`nvidia-smi`	`/proc/meminfo`	`/sys/class/power_supply`
Windows (NVIDIA)	`nvidia-smi`	CIM	CIM + WMI
macOS	`system_profiler` (name only)	`sysctl` + `vm_stat`	`pmset`

On macOS the chipset name is resolved once and cached, because `system_profiler` is slow and the value never changes, while memory and power are sampled on every cycle.

4.3 Dashboard

The React dashboard polls `/status` and `/pipeline`. It shows the topology, per-node measured hardware, the scheduler strategy with the exact weights it produced, and the placement `llama.cpp` reported. Fields the worker could not measure render as *Unavailable*, never as zero.

4.4 Tests

Fifteen unit tests cover the parts where a silent error would be most damaging: seven in Go and eight in Rust. They assert that unavailable values survive as `null` through serialisation, that `[N/A]` fields are not coerced to zero, that real `nvidia-smi` and `vm_stat` output parses correctly, that malformed output is rejected rather than half-read, that identical measurements always yield identical tensor weights, that a missing measurement produces the documented fallback instead of an invented weight, and that a heartbeat carrying no telemetry does not erase the previous snapshot.

Continuous integration was deliberately left out of scope. All checks are run locally: `go test`, `go vet`, `cargo test`, `cargo clippy` with warnings as errors, and a TypeScript production build with zero lint warnings.

5 Experimental setup

5.1 Machines

All three machines are personally owned consumer laptops on one domestic Wi-Fi network. Nothing was rented, and no datacentre hardware was involved.

Table 2: Hardware. Video memory figures are as reported by the machine itself through the sources in Table 1.

Role	Machine	Accelerator	GPU memory	System RAM
Controller	ASUS ROG, Linux	RTX 3050 Laptop, CUDA	4,096 MiB	15,234 MiB
Remote worker	ASUS TUF, Windows	RTX 4050 Laptop, CUDA	6,141 MiB	16,003 MiB
Remote worker	MacBook Air, macOS	Apple M4, Metal	unified	16,384 MiB

The Mac row carries no separate GPU memory figure because Apple Silicon has no separate pool to report. Its 16,384 MiB is one unified allocation shared between CPU and GPU.

5.2 Software

Both NVIDIA machines run `llama.cpp` release `b11909`, built with CUDA and RPC support, at RPC protocol 8.0.1. The Mac runs the official `b11909` macOS arm64 build. Keeping one release across all three machines removed protocol mismatch as a variable. Every remote `ggml-rpc-server` runs with `-c`, which caches received tensors on the worker's own disk so that only the first load transfers weights over Wi-Fi.

5.3 Models

- **Llama 3.2 1B Instruct**, Q4_K_M — local fallback profile.
- **Llama 3.2 3B Instruct**, Q4_K_M, 2.0 GB — used for every split and recovery experiment. It fits in the controller's GPU alone, which is what allows the controller to take over after a worker is lost.
- **Llama 3.1 8B Instruct**, Q4_K_M, 4.9 GB — capacity experiment. Verified byte-for-byte against the published artefact. It does not fit in the controller's 4,096 MiB GPU alone.

No weights are committed to the repository. Context length was 2,048 tokens throughout.

5.4 Network

A 5 GHz Wi-Fi network. Link conditions were recorded at measurement time because they are the dominant variable in every distributed figure here. During the benchmark runs of Section 6.3 the controller reported −56 dBm with driver-reported rates of 780 Mbit/s receive and 292.5 Mbit/s transmit. During the earliest model transfers, several days before, the same link reported −66 dBm to −68 dBm and 26 Mbit/s to 29 Mbit/s transmit. These are physical-layer rates negotiated by the driver, not achieved application throughput, and the gap between the two sessions explains why first-load times differ so widely in Section 6.5.

5.5 Method

Performance figures come from five sequential requests against an already-loaded model, using one fixed prompt asking for a 100-word explanation of distributed computing, capped at 160 output tokens. Normal sampling was left enabled, so output lengths vary slightly between runs; we therefore report medians and full ranges rather than a single best run. End-to-end latency is

`curl`'s `time_total`. Decode throughput is taken from `llama.cpp`'s own per-request timing line, using the `eval time` figure and excluding `prompt eval time`; it is never derived from response character counts.

Three configurations were measured:

1. **Local baseline** — `llama.cpp` on the controller alone, no RPC.
2. **Native split** — `llama.cpp` directly, with the Windows RPC endpoint and the same tensor weights HyperPod had computed.
3. **HyperPod split** — the same split reached through the Go controller.

Comparing the second and third isolates the control plane, since the inference path is identical. Failure experiments were performed by stopping a `ggml-rpc-server` process by hand while a request was in flight; no failures were simulated in software for the results reported here.

6 Results

6.1 Measured telemetry

Before any model was loaded, the workers reported the figures in Table 3. These are the values the scheduler consumes.

Table 3: Worker telemetry as received by the controller. Dashes mark fields the platform does not expose; these are transmitted as `null` and shown in the dashboard as *Unavailable*.

	TUF (Windows)	**M4 Mac**
GPU name	RTX 4050 Laptop	Apple M4
GPU memory, free / total	5,843 / 6,141 MiB	—
GPU utilisation	0 %	—
GPU temperature	50 °C	—
GPU power draw	4.09 W	—
GPU power limit	—	—
System RAM, available / total	4,845 / 16,003 MiB	5,938 / 16,384 MiB
Battery, AC	84 %, online	16 %, online

The Windows GPU power limit is blank because `nvidia-smi` returned `[N/A]` on that machine. The Mac's GPU rows are blank because macOS publishes no per-GPU memory, utilisation, temperature or power figure for Apple Silicon through the interfaces we use. Both cases are reported as unavailable rather than filled with a substitute.

6.2 Scheduling and actual placement

With both NVIDIA devices reporting free memory, the scheduler produced weights directly from measurement:

$$\underbrace{5843 - 512}_{\text{TUF RPC}} = 5331, \qquad \underbrace{3716 - 512}_{\text{ROG local}} = 3204, \qquad \Rightarrow \quad \texttt{-tensor-split 5331,3204}$$

Figure 2 shows the placement `llama.cpp` subsequently reported for four configurations. These are read back from its load output, not inferred from the request.

Two observations follow from the figure. In the three-machine configuration the Mac received the largest remote share, 968 MiB, even though HyperPod supplied no weights for it; that allocation was `llama.cpp`'s own decision under the documented fallback. And in the 8B run

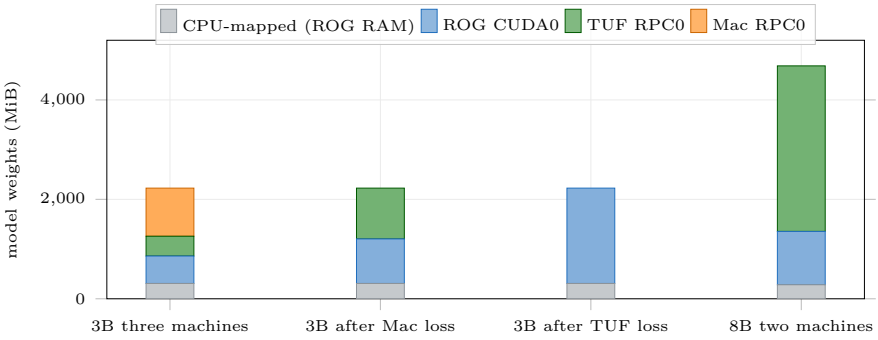


Figure 2: Per-device model weights reported by `llama.cpp` after loading, for four measured configurations. The first two bars are the same request before and after the Mac accelerator was stopped: its 968 MiB is redistributed onto the two surviving machines. The fourth bar is the 8B capacity run, where the remote Windows GPU holds 3,330 MiB of the 4.9 GB model.

the remote Windows GPU holds 3,330 MiB against 1,073 MiB on the controller. That is 71 % of all reported model weights, or 76 % of the weights resident on a GPU, and it is what makes that model runnable at all on this hardware.

6.3 Latency and throughput

Figure 3 gives all five runs for each configuration. We plot every run rather than only the medians, because the run-to-run spread is the most important feature of the distributed measurements.

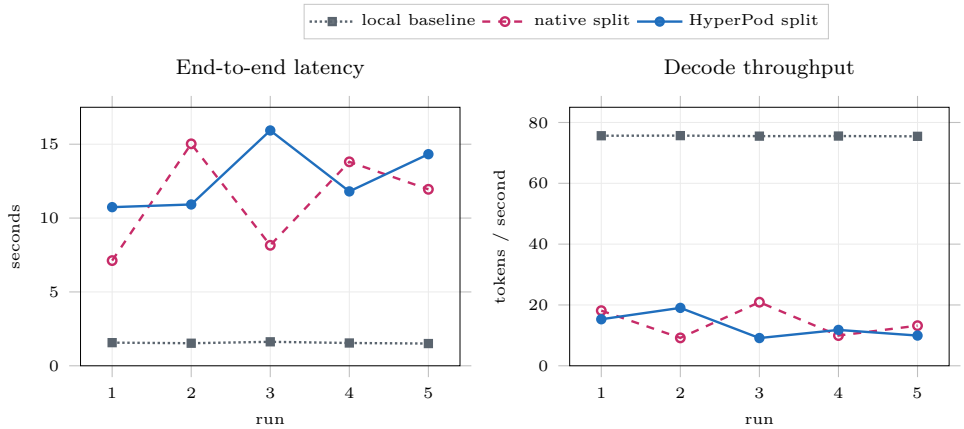


Figure 3: Five sequential requests per configuration against a warm model. The local baseline is tightly clustered, varying by 0.12 s and 0.23 tokens/second across all five runs. Both distributed configurations vary by several seconds and by more than a factor of two in throughput, which we attribute to the Wi-Fi link rather than to the control plane.

Table 4: Medians and ranges over the five runs of Figure 3.

Configuration	Median latency	Range	Median decode	Range
Local baseline	1.543 s	1.505–1.623	75.54	75.45–75.68
Native split	11.947 s	7.122–15.025	13.19	9.17–20.91
HyperPod split	11.804 s	10.739–15.930	11.79	9.12–19.04

The local baseline decoded at a median 75.54 tokens/second against 11.79 for the HyperPod split: distributing this model over this Wi-Fi network cost roughly a factor of six in throughput.

That is the central cost of the approach and we do not present it as anything else.

Between the two distributed configurations the median latencies differ by 0.143 s, which is far smaller than the spread within either configuration: 5.2 s for the HyperPod split and 7.9 s for the native split. With five runs we can state that the control plane introduced no latency penalty large enough to separate from link variance; we cannot make a stronger claim than that, and a larger sample would be needed to try.

6.4 Failure and recovery

Three failure experiments were run, all by stopping a real `ggml-rpc-server` process by hand during a live request. Figure 4 shows them on a common time axis.

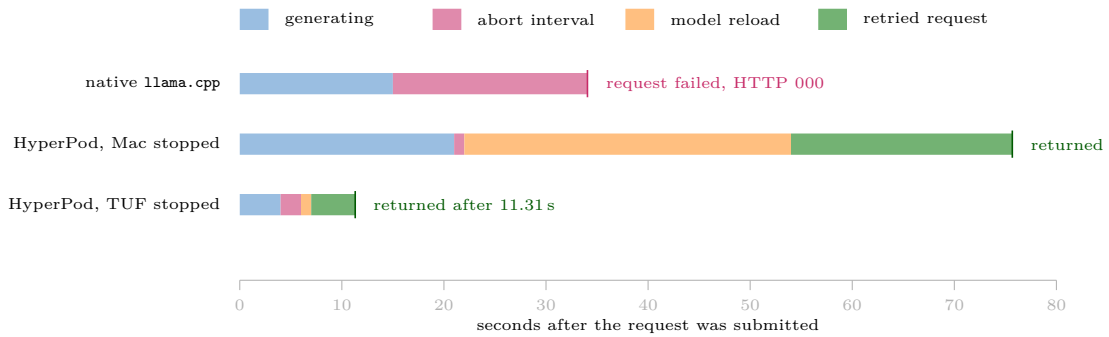


Figure 4: The three physical failure experiments. Blue is generation, red is the interval containing the abort, orange is model reload, green is the retried request. In the top run, unmodified `llama.cpp` never produced a response. In both HyperPod runs the original HTTP request completed without resubmission. Prompt lengths differed between runs, so the total durations are not comparable to each other; the comparison being made is outcome, not speed. The moment the operator stopped the native run's accelerator is known only to the nearest minute, so its red interval begins at an approximate offset; all other boundaries come from logged timestamps.

Native baseline. A request was submitted against the two-machine split and the Windows accelerator was stopped roughly fifteen seconds later. The client received no response: `curl` exited 52 with HTTP 000 after 34.06 s. The model process had aborted and did not return.

Three machines to two. The 3B model was loaded across all three machines and a request capped at 420 output tokens was submitted. The Mac's `ggml-rpc-server` was then stopped, while its Rust agent was deliberately left running. This separates two states that are easy to conflate: the agent continued to heartbeat normally, so agent health alone would have indicated a working node.

The controller log records the sequence:

```
llama-server gen 1 exited:  signal:  aborted
infer attempt 1 recognized node death and re-ran the same request
pool skipping 192.168.1.13:50052 (worker heartbeat ok, but its rpc-server is
unreachable)
launched gen 2 over [192.168.1.5:50052]; scheduler=measured-free-vram weights=4763,3204
pipeline ready (gen 2) after 33s
```

The rebuilt pipeline was still distributed, now holding 896 MiB on the controller GPU and 1,022 MiB on the Windows worker. The original request returned a 1,937-character response 75.68 s after submission.

Two machines to one. Repeating the experiment against the Windows worker produced the same outcome by a shorter path. Generation 1 aborted four seconds after submission, the unreachable endpoint was excluded, and because the 3B model fits in the controller's GPU alone the rebuild completed in about one second. The same request returned a 1,828-character response after 11.31 s in total.

6.5 Load times

Loading dominates the time cost of this design, and it varies with cache state and link quality rather than with anything HyperPod controls. Figure 5 collects the measured loads.

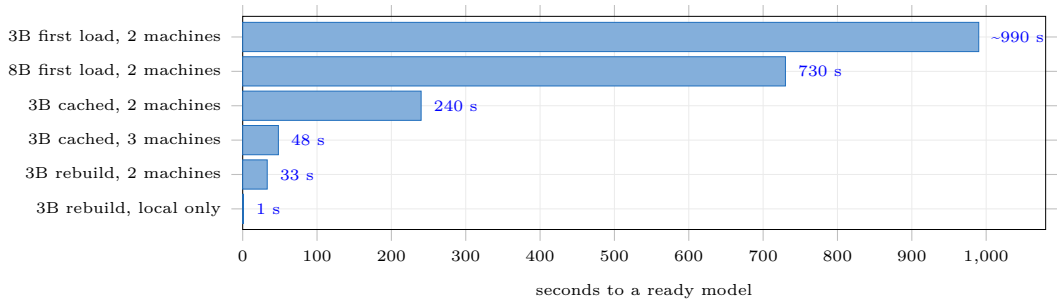


Figure 5: Measured time to a ready model. Each bar is a single observation, not an average. The two first-load bars include transferring the remote worker's share of the weights over Wi-Fi on a weak link (−66 dBm to −68 dBm); once the worker's local tensor cache is populated, the same 3B model reloads in a small fraction of that time. The cached three-machine load was measured on the stronger −56 dBm link.

The practical consequence is that the first load of a new model on a new worker is slow and should be done before any demonstration, while subsequent loads and all recoveries are short. Reported cached 3B loads across sessions were 240 s, 137 s, 60 s and 48 s, decreasing as link conditions improved; we report the range rather than selecting the most favourable figure.

7 Limitations

These are stated plainly because each one changes what the measurements in Section 6 mean.

It is slower, not faster. Distributing the 3B model over this Wi-Fi network cost roughly a factor of six in decode throughput against the single local GPU. HyperPod buys aggregate memory and recovery; it does not buy speed. Anyone whose model already fits on one machine should run it on one machine.

Recovery restarts, it does not resume. When a worker is lost, `llama.cpp` aborts and HyperPod starts a new model process and re-runs the request from the beginning. Tokens already generated are discarded and the KV cache is not preserved. The user sees one delayed response rather than a partial one, but the work is genuinely repeated. Continuing from the interrupted token would require state checkpointing inside the inference engine, which we did not attempt.

The 8B configuration is not fault tolerant on two machines. The 8B model does not fit in the controller's 4,096 MiB GPU alone, so losing the remote worker leaves nowhere for its 3,330 MiB share to go. That configuration demonstrates pooled capacity only. All recovery results in this report use the 3B model, which the controller can hold by itself.

The scheduler uses memory, not speed. Weights are derived from free video memory. Compute throughput, link bandwidth and link latency are measured and displayed but do not influence placement. A fast GPU behind a weak link therefore receives the same share as a fast GPU behind a strong one, which is not optimal.

Metal devices fall back to the engine's own placement. Apple Silicon reports no separate free-video-memory figure, so no comparable weight can be computed for it. Mixed CUDA and Metal clusters use `llama.cpp`'s own device probing, reported as `llama.cpp-auto`. The three-machine placement in Figure 2 was chosen by `llama.cpp`, not by HyperPod.

Joining does not trigger a rebuild. A worker that appears after a model is loaded is registered and displayed but is not added to the running pipeline. Rebuilds happen on failure, not on arrival. We hit this during our own screenshot session: a freshly started worker sat idle until the pipeline was rebuilt.

The controller is a single point of failure. Worker loss is handled; controller loss is not. If the controller stops, the cluster stops. Workers do re-register automatically when it returns, but in-flight work is lost.

No authentication or encryption. The `llama.cpp` RPC backend is documented upstream as insecure, and HyperPod adds no authentication of its own. The controller API, the worker endpoints and the discovery broadcast all assume a trusted local network. None of these ports should be exposed to the internet.

Discovery covers one broadcast domain. Workers are found by UDP broadcast, so they must share a subnet with the controller. Routed networks require explicitly configured endpoints.

The sample sizes are small. Performance figures are five runs per configuration on one network on one day. That is adequate to show a factor-of-six gap between local and distributed inference, and inadequate to resolve the 0.143 s median difference between native and HyperPod splits. Each load time in Figure 5 is a single observation. Each failure experiment was run once in the form reported.

8 Related work

HyperPod sits on top of existing work rather than replacing it, and the ideas it uses are not new. This section records what belongs to others.

`llama.cpp` and its RPC backend. All inference here is performed by `llama.cpp`: GGUF loading, quantised kernels, CUDA and Metal execution, and the RPC backend that exposes a remote accelerator. The ability to combine the memory of several machines is therefore upstream functionality, not ours. What upstream does not provide, and explicitly describes as out of scope for a proof-of-concept backend, is surviving the loss of a peer. That gap is what HyperPod addresses.

Consumer-hardware distributed inference. Running a single model across several personal devices has been explored by other projects, including work that schedules layers according to each device's capability and work that distributes models across volunteer machines over the public internet. Capability-aware placement in particular is well established; our scheduler is a

deliberately simple instance of it, using one measured quantity. We make no novelty claim for the idea of weighting devices by available memory.

Fault tolerance in serving systems. Detecting a failed node, excluding it, and restarting work on the survivors is standard practice in distributed systems and in datacentre model serving. The mechanisms used here — heartbeats, health timeouts, endpoint probing, process supervision, request retry — are all conventional. Their combination on three mismatched laptops over domestic Wi-Fi, with the resulting costs measured, is the part we contribute.

Meta open-weight models. The models are Meta's: Llama 3.2 1B and 3B, and Llama 3.1 8B, used as published GGUF quantisations. HyperPod does not fine-tune or modify them. Access to open weights is what makes the work possible at all, because the system moves model weights between physical devices and reads back where they landed. An API-only model exposes no such surface.

Where this work differs. Three things distinguish what is reported here from the surrounding work. The placement figures are read out of the inference engine's own load output rather than assumed from the requested split, which is how we found our tensor-order bug. Unavailable hardware values are preserved as unavailable all the way to the interface, so the Mac's missing video-memory figure is visible instead of being filled in. And the failure experiments were performed by stopping real processes on real machines during live requests, with the resulting logs committed, rather than by injecting faults in software.

9 Conclusion

HyperPod turns three mismatched consumer laptops into a single local runtime for Meta open-weight models, and keeps serving when one of them stops.

The measurements in this report establish four things on real hardware. A Llama 3.2 3B model was placed across a Linux RTX 3050, a Windows RTX 4050 and an M4 MacBook Air, with the per-device byte counts read back from `llama.cpp` rather than assumed. Stopping the Mac accelerator mid-request caused the inference process to abort, after which HyperPod excluded the unreachable endpoint, rebuilt a still-distributed pipeline across the two survivors in 33 s, and returned the same HTTP request without resubmission; under the same conditions unmodified `llama.cpp` failed the request and did not restart. A Llama 3.1 8B model that does not fit in the controller's 4,096 MiB GPU ran with 3,330 MiB of its weights on the remote Windows GPU. And distributing over Wi-Fi cost roughly a factor of six in decode throughput relative to the single local GPU.

The last of those is as important as the first three. This design trades throughput for capacity and for recovery. On a network like ours the trade only makes sense when the model does not fit on one machine, or when an interrupted request matters more than its latency.

What we would build next follows from Section 7. Caching model weights on each worker's disk ahead of time would remove the long first load, which is currently the worst part of the experience. Adding measured compute and link bandwidth to the scheduler would let it place work on the machine that can actually do it fastest, rather than the one with the most free memory. Preserving the KV cache across a rebuild would turn restart-and-retry into genuine continuation. Controller election would remove the last single point of failure. Authentication would make the system safe outside a trusted network.

Every figure quoted here is reproducible from the committed logs under `test/` in the repository, and the two physical failure experiments are recorded in `test/phase6-three-node-recovery.log` and `test/phase7-native-benchmark.log`.

Built with Llama. Llama 3.1 and Llama 3.2 are licensed by Meta under their respective Community License Agreements. `llama.cpp` is used under the MIT License. Full notices are in `THIRD_PARTY_NOTICES.md`.